TSDS: Auditable Final-Byte Evidence for Command-Injection Triage in Edge Firmware

Zuozheng Zhou, Jian Lin*, Shuai Ren, Haoran Liu, Jintao Bao, and Ruimin Wang
Information Engineering University, China
Email: {zzczzz_1amfree, renshuai94, Cherest_San}@outlook.com, {ling_pro, taoxinying2022, wrm2013nian}@163.com
*Corresponding author: ling_pro@163.com

***Abstract*—Centralized DevSecOps pipelines routinely ingest edge gateway firmware before deployment. However, static analyzers emit hundreds of unranked command-injection warnings that overwhelm security operations. To resolve this triage bottleneck, we present TSDS. Built around an executable *final-byte evidence contract*, TSDS binds sink argument semantics, reconstructs concrete C strings, tracks byte provenance, and evaluates an 11-dimensional shell effect matrix.**

Across eight commercial router firmware images spanning 518 static warnings, TSDS isolates 36 complete-string and 5 observed-prefix injection profiles. This triage eliminates 92.1% of manual review volume. All 326 unresolved cases remain preserved as explicit typed obligations. Dynamic QEMU system emulation reproduces 6 known CVEs alongside 5 safe boundary cases at unquoted shell sinks. On a 24-case controlled benchmark, TSDS achieves $F_1 = 1.0$. The framework eliminates 10 reachability false positives and 5 regex false positives, and emits exact byte extents, deterministic quote contexts, and cryptographic audit receipts for cloud compliance pipelines without claiming automated exploit synthesis.

***Index Terms*—Cloud computing, edge security, IoT firmware analysis, alert triage, symbolic execution, command injection, evidence contracts**

I. Introduction

Modern cloud platforms (e.g., AWS IoT Core, Azure IoT Edge) centrally manage distributed edge gateways, where commercial router firmware mediates operational telemetry for local device clusters. In cloud continuous deployment (DevSecOps) workflows, cloud operators must continuously vet edge firmware updates against supply-chain vulnerabilities before pushing images to distributed edge nodes. However, edge firmware frequently invokes privileged shell commands via `system`, `popen`, and vendor wrappers. When cloud SecOps teams vet firmware images using static taint scanners [1]–[6], analysts face hundreds of unranked warnings. Because static taint closures cannot verify whether tainted bytes reach sinks unquoted or execute shell syntax, operators suffer severe triage fatigue; this backlog stalls automated compliance verification. While rehosting and symbolic execution provide deeper modeling [7]–[11], whole-firmware emulation fails on missing hardware peripherals and path explosion.

A static closure merely relates a source and sink, without identifying dependent final bytes, establishing callee shell semantics, or proving that a feasible source activates shell syntax. A taint bit or metacharacter cannot encode this chain. To resolve this operational bottleneck, TSDS introduces an *evidence-driven triage framework* for cloud-managed edge devices. By producing machine-checkable, digest-bound final-byte contracts before firmware ingestion, TSDS bridges candidate discovery (e.g., SaTC, Mango, CodeQL) and actionable cloud verification.

Unlike standard targeted symbolic execution that only checks if a sink instruction is reachable under path condition Φ, TSDS executes a structured 5-stage triage pipeline: *warning ingestion* $\rightarrow$ *sink argument binding* $\rightarrow$ *final-byte reconstruction* $\rightarrow$ *quote-aware matrix evaluation* $\rightarrow$ *cryptographic ledger emission*. Rather than detect more candidates, TSDS evaluates whether untrusted source bytes in the final command buffer can successfully escape quote contexts and trigger shell command execution. It applies four non-escalation admission rules: front-end closure is not observed final-byte dependence; conditioned template feasibility is not source linkage; projected SAT is rechecked under the full path store; and incomplete search yields an explicit obligation rather than a negative conclusion. TSDS enforces this evidence model via targeted symbolic execution with an 11-dimensional shell effect matrix that defaults to fail-closed for unmodeled syntax. Figure 1 shows the contract on one complete command instance.

a) Two-Tiered Evaluation & Practical Impact: We evaluate TSDS using a two-tiered methodology. First, on a 24-case controlled reference suite modeling distinct sanitization, quote escaping, and truncation primitives (inspired by NIST SARD CWE-78), TSDS correctly resolves all 24 ground-truth cases ($F_1 = 1.0$). This test eliminates false positives from raw reachability and regex scanning by isolating exact byte extents and deterministic quote contexts. Second, across eight commercial router firmware images spanning 518 static warnings, TSDS isolates 36 complete-string and 5 observed-prefix injection profiles. This triage reduces manual SecOps review volume by 92.1% while preserving all 326 unresolved cases as explicit typed obligations. Dynamic execution under QEMU system emulation confirms that synthesized command witnesses successfully reach unquoted shell sinks to reproduce 6 known CVEs and 5 boundary cases in these routers. TSDS establishes shell syntax feasibility strictly at the reached callee rather than claiming automated device-level exploit generation or privilege escalation. With a median triage time of 9.09 s per warning, TSDS establishes a compliance-supporting evidence pipeline without full-firmware rehosting bottlenecks.

b) Contributions: We make three contributions to cloud and edge security analysis:

1) An executable *final-command-byte evidence model* with explicit admission rules, distinguishing final-byte syntactic control from reachability closures.
2) *Evidence-directed symbolic execution* that integrates sink-corridor scheduling, semantic argument binding, byte provenance, and a quote-aware shell-effect matrix to verify syntax transitions.
3) A content-bound ledger and cryptographically verifiable audit record for cloud-compatible batch triage. SHA-256 digests uniquely separate automated evidence proofs (e.g., 6 real-world CVE reproductions) from explicit residual manual obligations to establish multi-team auditable trust and compliance-supporting evidence without transferring raw firmware.

II. Evidence Contract

A. Analysis Boundary and Threat Model

The input to TSDS is a binary and a front-end closure $c = \langle x, t, k \rangle$, where x identifies a candidate source, t is an optional interprocedural trace, and k identifies a command sink. The front end supplies search guidance, not outcome evidence. Candidate documents are normalized under a schema that rejects leaked verdicts, controlled offsets, or matrix decisions. A digest binds the original closure, binary, source and sink addresses, and front-end identity.

The modeled adversary controls bytes returned by configured Web, NVRAM, configuration, environment, file, and entry-argument summaries. The query asks whether those bytes can activate declared shell syntax at a claimable sink. Authentication, routing, deployment state, and post-shell effects are included only when materialized as constraints or command bytes. TSDS verifies command-byte shell syntax feasibility at the reached callee, not device-level post-exploitation or weaponized exploit chains.

B. Final-Byte Semantics

For a reached state s, write $B = B(s) = \langle b_0, \ldots, b_{n-1} \rangle$ for the captured command C string up to the first verified terminator τ_s. Let X_c denote the symbolic variables introduced by the admitted source model for c. The source-controlled offset set is

$$I_{X_c}(B) \triangleq \{\, i \in [0, \tau_s) \cap \mathbb{N} \mid \mathrm{Vars}(b_i) \cap X_c \neq \varnothing \,\}. \quad (1)$$

The terminator may have source lineage for provenance purposes, but it is not an active shell-vector offset.

A captured argument is claimable only if both callee semantics and argument binding are explicit: $\mathrm{Claimable}(s) \triangleq \mathrm{Sem}(s) \in \mathcal{S}_{\mathrm{sh}} \wedge \mathrm{Trust}(s) \in \mathcal{T}_{\mathrm{arg}}$, where $\mathcal{S}_{\mathrm{sh}} = \{\mathsf{shell_command}, \mathsf{shell_format_wrapper}\}$ and $\mathcal{T}_{\mathrm{arg}} = \{\mathsf{direct_abi_arg0}, \mathsf{rendered_format_wrapper}\}$. Direct `exec` calls and unknown wrappers require different interpreter models and therefore remain non-claimable under this contract.

The dynamic unit is a selected command instance, not the complete path space of c. Write $\mathrm{Ready}(s) \triangleq \mathrm{Claimable}(s) \wedge \mathrm{CompleteCString}(B(s))$, and define the complete direct-ready premise $\mathsf{DR}(s) \triangleq \mathrm{Ready}(s) \wedge \mathrm{Mode}(s) = \mathrm{D}$. Let $\mathcal{H}_c^{(b)}$ be the Ready sink states retained by the bounded collector. A separate $\mathrm{PrefixReady}(s)$ predicate denotes a claimable source-bearing prefix whose terminator or suffix is not verified; such a view is retained as a weaker sidecar and is excluded from the complete direct aggregate. Let $\mathcal{H}_{c,\mathrm{P}}^{(b)}$ contain those prefix-only views. After matrix evaluation, $\mathrm{PosComplete}(s)$ denotes a direct-ready state with a complete matrix and at least one SAT cell. Let $\mathrm{AdmissibleMatrix}(s)$ denote a complete, claimable direct matrix profile, including non-positive profiles. For nonempty $\mathcal{H}_c^{(b)}$, the primary selector first prefers the earliest positive profile, then the earliest admissible complete profile, and otherwise returns $\perp$ rather than promoting an incomplete or non-admissible snapshot:

$$\begin{aligned} s_c &\triangleq \mathrm{Select}_b(\mathcal{H}_c^{(b)}, \prec_c), \\ \mathcal{P}(\mathcal{H}) &\triangleq \{s \in \mathcal{H} \mid \mathsf{DR}(s) \wedge \mathrm{PosComplete}(s)\}, \\ \mathcal{Q}(\mathcal{H}) &\triangleq \{s \in \mathcal{H} \mid \mathsf{DR}(s) \wedge \mathrm{AdmissibleMatrix}(s)\}, \\ \mathrm{Select}_b(\mathcal{H}, \prec) &\triangleq \begin{cases} \min_\prec \mathcal{P}(\mathcal{H}), & \mathcal{P}(\mathcal{H}) \neq \varnothing, \\ \min_\prec \mathcal{Q}(\mathcal{H}), & \mathcal{P}(\mathcal{H}) = \varnothing \wedge \mathcal{Q}(\mathcal{H}) \neq \varnothing, \\ \perp, & \text{otherwise}, \end{cases} \\ \rho_c &\triangleq \langle c, s_c, B_c, \Phi_c, \kappa_c \rangle, \quad s_c \neq \perp. \end{aligned} \quad (2)$$

For $s_c \neq \perp$, $B_c = B(s_c)$, $\Phi_c = \Phi_{s_c}$, and $\kappa_c = \kappa_{s_c}$. If $\mathcal{H}_c^{(b)} = \varnothing$, the selector is bypassed and Algorithm 1 continues to a static or residual record. If the selector returns $\perp$, the collected snapshots remain audit evidence but cannot drive the primary semantic classification. The explicit `-first-ready-legacy` option replaces Select_b by the first Ready state for compatibility and ablation. Equation 2 is a deterministic selection rule, not an assertion that s_c is the only reachable sink state. Every dynamic claim below is scoped to ρ_c.

Let $\mathcal{V}$ be the declared 11-vector matrix. Predicate $P_v(B_c, I_{X_c}(B_c), \chi_c)$ represents vector v under quote context χ_c and constrains syntax only at source-controlled offsets. The authoritative decision has a three-valued domain:

$$\begin{aligned} \mathbb{D} &\triangleq \{\mathsf{SAT}, \mathsf{UNSAT}, \mathsf{INC}\}, \\ Q_v(\rho_c) &\triangleq \mathrm{Solve}_{\mathbb{D}}(\Phi_c \cup \{P_v(B_c, I_{X_c}(B_c), \chi_c)\}). \end{aligned} \quad (3)$$

$\mathrm{Solve}_{\mathbb{D}}$ returns INC when no complete placement exists or a context or solver obligation is unresolved. A SAT model is serialized as a minimal command-byte witness; UNSAT remains scoped to ρ_c, the source model, and the versioned matrix.

C. Evidence Classes and Claims

Figure 1 instantiates the contract on Netgear XR300 (`todo=qos_dev_edit`). An unauthenticated Web request formats an interface name into `tc qdisc del dev <dev> root` before invoking `system` at `0x19a48`. Static prefix bytes 0–16 establish an unquoted context (S_{unq}) at offset 17. The adversary controls 31 bytes (offsets 17–47). Command substitution `$(:)` and chaining (`;`, `&`, `|`) are satisfiable (SV-SAT), with minimal witness `tc qdisc del`

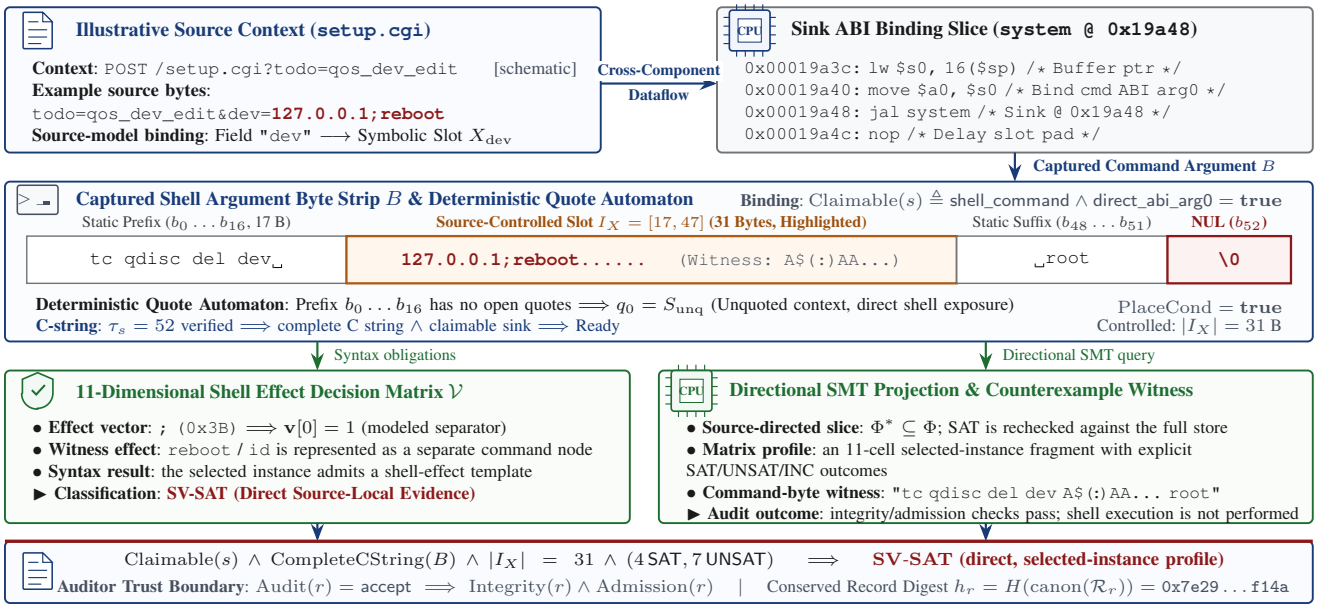

Fig. 1. Worked final-byte contract for XR300 closure 30. The request and assembly snippets are schematic context; the selected sink instance has 31 controlled offsets, a 4-SAT/7-UNSAT profile, and a digest-bound direct SV-SAT record. The figure reports no shell execution or device-level trigger.

`dev A$(:)AA... root` and a 4-SAT/7-UNSAT profile. The contract binds these offsets, quote states, decisions, and witness under a SHA-256 digest for zero-recomputation cloud audit. A SV-SAT profile is *partial* when it also contains UNSAT or inconclusive cells. For a directly observed complete instance ρ belonging to candidate c, the source-variable set is the candidate-bound set X_c from Eq. 1. Define the explicit direct-ready premise $\mathsf{DR}(\rho) \triangleq \mathrm{Mode}(\rho) = \mathrm{D} \wedge \mathrm{Claimable}(\rho) \wedge \mathrm{CompleteCString}(B_\rho)$; for compactness, let $\mathsf{A}_{\mathrm{D}}(\rho) \triangleq \mathsf{DR}(\rho) \wedge I_{X_c}(B_\rho) \neq \varnothing \wedge \mathrm{CM}(\rho)$ be the complete direct-matrix premise. The three dynamic predicates are

$$\begin{aligned} \mathrm{SV_D}(\rho) &\iff \mathsf{A}_{\mathrm{D}}(\rho) \wedge \exists v \in \mathcal{V} \;:\; Q_v(\rho) = \mathsf{SAT}, \\ \mathrm{MF}(\rho) &\iff \mathsf{A}_{\mathrm{D}}(\rho) \wedge \forall v \in \mathcal{V} \;:\; Q_v(\rho) = \mathsf{UNSAT}, \\ \mathrm{NMS}(\rho) &\iff \mathsf{DR}(\rho) \wedge I_{X_c}(B_\rho) = \varnothing. \end{aligned} \tag{4}$$

For a prefix-only view ρ^{P}, let B^{P} stop at the last observed byte and let CM^{P} mean that the declared vectors were evaluated only over that prefix. Define $\mathsf{A}_{\mathrm{P}}(\rho^{\mathrm{P}}) \triangleq \mathrm{PrefixReady}(\rho^{\mathrm{P}}) \wedge I_{X_c}(B^{\mathrm{P}}) \neq \varnothing \wedge \mathrm{CM}^{\mathrm{P}}(\rho^{\mathrm{P}})$. Its weaker positive predicate is

$$\text{P-SAT}(\rho^{\mathrm{P}}) \iff \mathsf{A}_{\mathrm{P}}(\rho^{\mathrm{P}}) \wedge \exists v \in \mathcal{V} \;:\; Q_v^{\mathrm{P}}(\rho^{\mathrm{P}}) = \mathsf{SAT}. \tag{5}$$

The prefix-local result does not establish a complete-string SV-SAT profile. For the 5 observed-prefix records, the terminator was not observed, their suffix is unknown, and their quote context has a truncation boundary. Therefore, they do not inherit the syntactic inference privileges of the 36 complete-string SV-SAT profiles, and cannot support M-FILT or NMS. Complete SV-SAT carries an existential witness. M-FILT and NMS are matrix-local and source-model-local descriptors of ρ; they do not quantify over other sink instances and are never candidate-wide negative conclusions. Conditioned reconstruction uses a separate class, CT-SAT, because satisfiability over a fresh slot does not prove an admitted input can reach that slot. M-FILT means "matrix-UNSAT at the selected instance," not warning elimination. Table I summarizes this vocabulary. When no dynamic instance completes, strong source-slot evidence creates a static source obligation, while a fixed template without a source slot creates a template observation. Neither rule creates controlled offsets or matrix decisions.

TABLE I
RECORD VOCABULARY AND LOGICAL SCOPE. CONDITIONED TEMPLATE FEASIBILITY IS SEPARATED FROM OBSERVED FINAL-BYTE DEPENDENCE.

Record	Unit	Admission	Logical force
SV-SAT	Inst. (direct)	Complete; obs. extent; $\exists v$ SAT	Existential complete-string vector
P-SAT	Inst. (prefix)	Prefix only; $\exists v$ SAT	Prefix-local positive; no suffix claim
CT-SAT	Inst. + tpl.	Reconstructed slot; $\exists v$ SAT	Conditioned template feasibility
M-FILT	Inst. (direct)	Complete; $\forall v$ UNSAT	11-vector local profile
NMS	Inst. (direct)	Complete; $I_X = \varnothing$	Source-model-local snapshot
Static src	Candidate	Strong source slot; open	Follow-up obligation
Static tpl	Candidate	Fixed template; no slot	Template observation
RESIDUAL	Candidate	Unresolved boundary	No semantic claim

a) Conservation invariant and soundness boundary: Let $\mathcal{C}$ be the normalized candidates, let $\mathcal{R}_c$ contain the modeled Ready instances for candidate c, and let $\mathcal{L}$ be the record classes in Table I, including the prefix-only class P-SAT. We write $\mathrm{Scope}(\cdot)$ for the instances to which a record is allowed to speak. Algorithm 1 and the admission rules enforce

$$\begin{gathered} \textstyle\sum_{\ell \in \mathcal{L}} \mathbf{1}[\mathrm{Class}(c) = \ell] = 1, \\ \mathrm{SV_D}(\rho_c) \Rightarrow \exists \rho \in \mathcal{R}_c \;:\; \mathrm{SV_D}(\rho), \\ \mathrm{Scope}(\mathcal{K}(\rho_c)) = \{\rho_c\} \subseteq \mathcal{R}_c, \quad \mathcal{K} \in \{\mathrm{MF}, \mathrm{NMS}\}. \end{gathered} \tag{6}$$

The first line proves the *Candidate Conservation Invariant* (no warning is dropped or double-counted). The remaining lines establish the *Soundness Boundary*: an existential positive witness proves satisfiability at a modeled instance, whereas local non-positive profiles hold strictly over their selected instance with no false promotion to candidate-wide safety.

For the bounded collector, let $\mathcal{H}_c^{(b)} \subseteq \mathcal{R}_c$ be the sink instances collected during a configured settle window, and let $\mathrm{Complete}_b$ mean that the bounded execution frontier, rather than the complete program path space, was exhausted. The producer records this bit together with seed and engine accounting; an explicit incomplete accounting bit or any blocker forces $\mathrm{Complete}_b$ to be false. We separate the observed direct instances from any conditioned sidecar profiles:

$$\mathcal{H}_{c,\mathrm{D}}^{(b)} \triangleq \{\rho \in \mathcal{H}_c^{(b)} \mid \mathrm{Mode}(\rho) = \mathrm{D}\}.$$

Here D denotes observed source provenance at complete sink bytes. Prefix-only views are written $\mathcal{H}_{c,\mathrm{P}}^{(b)}$ and conditioned profiles are written $\mathcal{H}_{c,\mathrm{C}}^{(b)}$; both are retained as separately scoped sidecars. Only complete $\mathcal{H}_{c,\mathrm{D}}^{(b)}$ participates in the primary bounded aggregate. A positive prefix sidecar may be serialized as P-SAT, but it cannot satisfy Pos_D. Let $\mathrm{CompleteMatrix}(\rho)$ mean that all 11 declared vector identities have a decision in $\mathbb{D}$, and let $\mathrm{CM}(\rho)$ be its abbreviation. Define $\mathrm{Pos}_\mathrm{D}(\rho)$ as a positive direct profile. The abbreviation $\mathrm{AllUNSAT}(\rho)$ below means a complete matrix profile in which every declared vector is explicitly UNSAT; an INC cell is not negative evidence. The bounded aggregation uses an existential rule for positive evidence and a non-escalating rule for non-positive observations. For compactness below, write $\mathcal{H}_D$ for $\mathcal{H}_{c,\mathrm{D}}^{(b)}$ and $A_b(c)$ for $\mathrm{Complete}_b \wedge \mathcal{H}_D \neq \varnothing$. We distinguish complete matrix-UNSAT from complete no-source profiles, and abbreviate the four outcomes as EP (exists-positive), ANP (all-collected matrix-UNSAT), ANS (all-collected no-source), and BI (bounded-incomplete):

$$\begin{aligned}
\mathrm{Pos}_\mathrm{D}(\rho) &\iff \mathsf{DR}(\rho) \wedge \mathrm{CM}(\rho) \wedge \exists v \in \mathcal{V} \,:\, Q_v(\rho) = \mathsf{SAT},\\
\mathrm{Pos}_b(c) &\iff \exists \rho \in \mathcal{H}_D \,:\, \mathrm{Pos}_\mathrm{D}(\rho),\\
\mathrm{NonPos}_b(c) &\iff A_b(c) \wedge \forall \rho \in \mathcal{H}_D \,:\, \mathrm{AllUNSAT}(\rho),\\
\mathrm{NoSource}_b(c) &\iff A_b(c) \wedge \forall \rho \in \mathcal{H}_D \,:\, \mathrm{NMS}(\rho),\\
\mathrm{Agg}_b(c) &\triangleq \begin{cases} \mathsf{EP}, & \mathrm{Pos}_b(c),\\ \mathsf{ANP}, & \mathrm{NonPos}_b(c),\\ \mathsf{ANS}, & \mathrm{NoSource}_b(c),\\ \mathsf{BI}, & \text{otherwise},\end{cases}\\
\mathsf{ANP} \vee \mathsf{ANS} &\Rightarrow \mathrm{Scope}(\mathrm{Agg}_b(c)) = \mathcal{H}_D \subseteq \mathcal{R}_c.
\end{aligned} \tag{7}$$

The bounded aggregation record is therefore an explicit collected-instance summary. It can recover a positive instance missed by a first-hit-only observation within the configured window, but it cannot turn a finite non-positive collection into a candidate-wide safety or filtering claim. The current evaluator uses this bounded policy by default; the published V20 ledger is a frozen historical run and is not retroactively relabeled. The legacy first-Ready mode remains available only for compatibility and ablation.

Network/device reachability, post-shell effects, universal sanitizer correctness, and source-model completeness remain explicit downstream confirmation dimensions rather than implicit ledger claims.

D. Integrity-Bound Evidence Record

For record r, let L_r be its normalized source-ledger row, $I_r = I_{X_c}(B_r)$, and M_r^{sink} denote sink metadata; let $\mathbf{d}_r$ serialize the possibly empty sequence of instantiated triples $(v, Q_v(\rho_r), w_v)$. The canonical payload and record digest are:

$$\begin{aligned}
\mathcal{R}_r &\triangleq \big\langle id_r, H(L_r), claim_r, I_r, M_r^{\mathrm{sink}},\\
&\qquad H(\mathcal{V}), \mathbf{d}_r, recovery_r, conditioning_r \big\rangle,\\
h_r &\triangleq H\big(\mathrm{canon}(\mathcal{R}_r)\big).
\end{aligned} \tag{8}$$

M_r^{sink} stores reachability, binding trust, and snapshot metadata; H is SHA-256 over canonical JSON, and $w_v = \perp$ denotes no witness. The source-ledger digest binds candidate, binary, command projection, provenance, and analyzer metadata. Without importing the analyzer, a standalone auditor checks serialization, offsets, matrix/witness completeness, sink admission, recovery, and conditioning. It establishes record integrity and admission consistency. The auditor's trust boundary is

$$\begin{aligned}
\mathrm{Audit}(r) = \mathsf{accept} &\Rightarrow \mathrm{Integrity}(r) \wedge \mathrm{Admission}(r),\\
\mathrm{Checked}_{\mathrm{Audit}}(r) &= \{\mathrm{Integrity}(r), \mathrm{Admission}(r)\}.
\end{aligned} \tag{9}$$

The record is therefore an integrity-bound audit object; independent replay or an UNSAT proof remains a separate semantic-verification layer.

III. System Overview

Figure 2 illustrates TSDS within a cloud DevSecOps pipeline for edge firmware. Designed as an elastic, containerized cloud service, TSDS ingests firmware images and static warnings from cloud registries and resolves four evidentiary gaps left unaddressed by static scanners. A digest-checked adapter binds candidate identity without inheriting front-end verdicts. Evidence search steers execution along the recovered sink corridor across distributed worker pods. A semantic plugin then resolves the callee contract and materializes the concrete command buffer to restrict matrix solving exclusively to source-controlled offsets. The ledger serializes all resulting instance profiles and open obligations under a single cryptographic identity to prevent unranked static warnings from entering cloud deployment pipelines as unverified claims.

Algorithm 1 enforces the candidate accounting invariant: every admitted candidate produces exactly one record. ResolveByteView returns an admitted direct or conditioned mode, or an unresolved byte-view obligation; other search, capture, binding, provenance, or solver failures remain strictly typed. A complete direct instance with no controlled extent is classified as NMS before matrix solving. A prefix-only view can emit the weaker P-SAT sidecar from Eq. 5, but cannot emit complete-string SV-SAT, M-Filt, or NMS. Unresolved views cannot enter semantic classes. Write $\mathrm{Ready}(\kappa, B) \triangleq \mathrm{Claimable}(\kappa) \wedge \mathrm{CompleteCString}(B)$.

The bounded collector visits sink hits in scheduler order and retains complete Ready and prefix-only views through the configured settle window. A deterministic primary selector

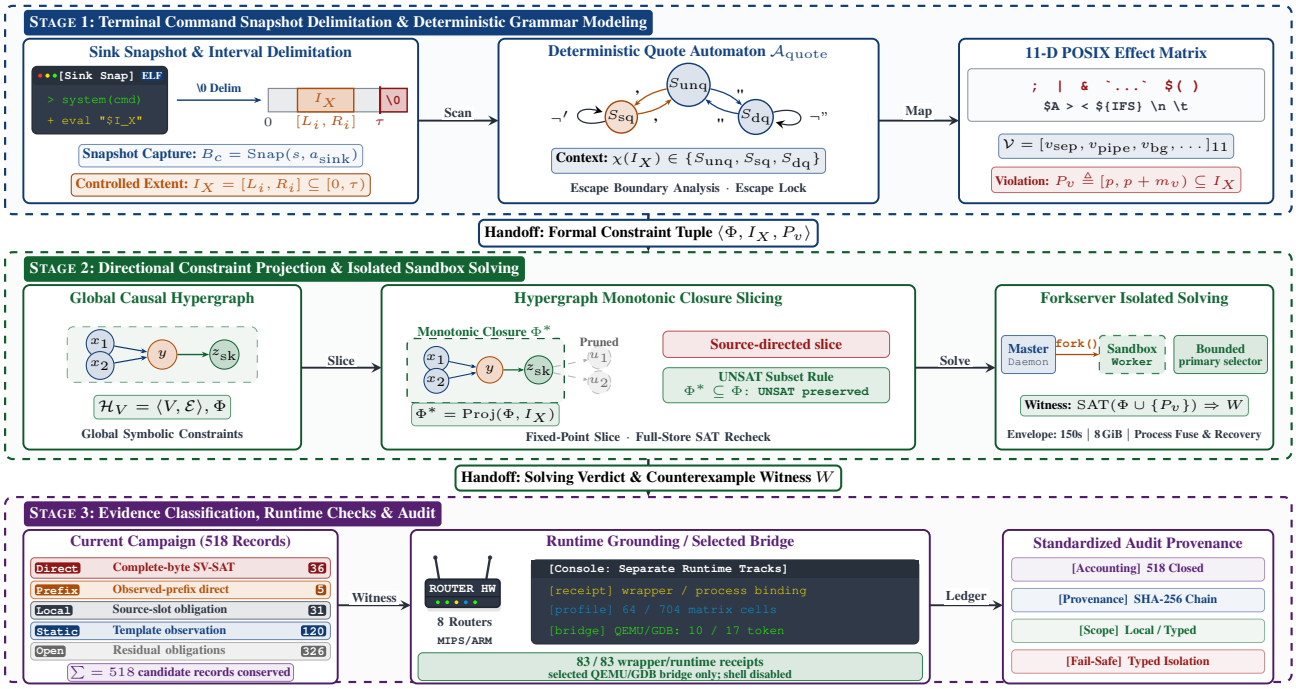

Fig. 2. TSDS realizes a final-byte evidence contract across candidate binding, evidence-directed search, semantic command reconstruction, and effect-local solving. The outcome card reports the current bounded campaign: 36 complete-byte direct profiles, 5 observed-prefix profiles, 31 source-slot obligations, 120 template observations, and 326 typed residuals. The runtime panel separates 83/83 wrapper/runtime receipts from the 10/17 QEMU/GDB selected-token observations; these auxiliary tracks do not imply device-level exploitability. Refinement and record auditing consume the conserved ledger without bypassing its admission path.

prefers the earliest complete positive profile, followed by the earliest complete claimable direct matrix profile. If neither exists, positive prefix sidecars are emitted strictly as P-SAT; otherwise, the candidate is conserved as a typed residual. No incomplete snapshot is ever promoted to complete direct evidence. In complete direct mode, a SAT cell supplies an existential witness for that modeled instance. In contrast, M-FILT and NMS remain local profiles and never authorize candidate-wide elimination. CT-SAT retains the same selected-instance scope alongside the reconstruction-link gate in Eq. 12. States outside the captured instance are never silently universalized: absent an admissible instance, the candidate is conserved as a static obligation or residual.

Guarded refinement synthesizes summaries exclusively for residuals. Each admitted bundle configures a bounded replay through Algorithm 1; the result remains conditioned and outside the primary aggregate.

IV. DESIGN

A. Candidate-Bound Context Reconstruction

TSDS validates a reported closure rather than rediscovering arbitrary flows. Its candidate contract binds the front-end and record digests, source/sink addresses, trace, inputs, and sink name while rejecting outcome fields. Duplicate identities are rejected; distinct front-end closures remain auditable. Because traces often begin without a caller frame, architecture-aware seed families initialize ARM argument/callee-saved carriers or MIPS `a0-a3`, `t9`, `gp`, and `ra`; main-like entries receive bounded `argc`/`argv` memory. Recorded seed assumptions must still reach the declared sink and satisfy the contract.

Firmware-aware summaries preserve evidence-relevant effects: sources return labeled strings; string summaries model copies, concatenation, tokenization, and bounded transfer; and format summaries retain fixed command fragments while binding only source-derived slots. Numeric conversions preserve lineage when their byte representation becomes constrained.

Algorithm 1 presents the current bounded collection policy. EVALUATEINSTANCE applies the complete-direct, prefix-only, or conditioned byte-view gate, the projected query rule, and full-store validation before a profile enters $\mathcal{H}_D$ and $\mathcal{H}_P$ through the settle budget; SELECTPRIMARY implements Eq. 2 and admits only complete, claimable direct matrix profiles. If none is collected, a positive prefix sidecar is emitted only as P-SAT; otherwise the algorithm emits a typed residual. AGGREGATEBOUNDED implements Eq. 7. The explicit `-first-ready-legacy` mode returns at the first Ready snapshot only for compatibility and ablation. The frozen V20 ledger is a historical first-Ready run and is reported as such rather than silently regenerated under the current default. A later positive snapshot can therefore be exposed within the bounded window, while an incomplete or non-positive collection cannot be promoted to a candidate-wide negative result.

B. Sink Corridor and Evidence-Aware Scheduling

A front-end trace is incomplete around indirect calls and wrappers. TSDS merges it with a recovered CFG to form a closure-specific *sink corridor*. Reverse search from the sink

Algorithm 1 Contract-guided evidence construction

Require: candidate c, binary F, matrix $\mathcal{V}$
policy $b \in \{\text{BOUNDED}, \text{LEGACY}\}$
Ensure: one candidate-bound ledger record r

Phase I: candidate binding
$\hat{c} \leftarrow \text{NormalizeAndBind}(c, F)$
if $\neg\text{ValidCandidate}(\hat{c})$ **then**
  return $\text{RecordResidual}(\hat{c}, \text{InvalidCandidate})$
$S \leftarrow \text{SeedStates}(F, \hat{c})$; $\mathcal{H}_D, \mathcal{H}_P \leftarrow \varnothing$
$O \leftarrow \text{NewObligation}(\hat{c})$

Phase II: bounded search and instance collection
while $S \neq \varnothing$ and budgets remain **do**
  $S \leftarrow \text{ScheduleCorridor}(S, \hat{c})$
  $S \leftarrow \text{Step}(S)$
  Sort sink hits in scheduler order $\prec_c$
  for all $s \in \text{SinkHits}(S)$ **do**
    $(\kappa, B) \leftarrow \text{BindAndCaptureSink}(s, \hat{c})$
    if $\text{Ready}(\kappa, B)$ or $\text{PrefixReady}(\kappa, B)$ **then**
      $(\widetilde{B}, I_X, G, \mu) \leftarrow \text{ResolveByteView}(s, B, \hat{c})$
      if $\mu = \bot$ **then**
        $O \leftarrow \text{UpdateObligation}(O, \text{ByteView})$
      else
        $\rho_s \leftarrow \langle \hat{c}, s, \widetilde{B}, \Phi_s, \kappa, \mu \rangle$
        $\mathbf{d}_s \leftarrow \text{EvaluateInstance}(\rho_s, I_X, G, \mathcal{V})$
        if $\text{CompleteCString}(\widetilde{B})$ **then**
          $\mathcal{H}_D \leftarrow \mathcal{H}_D \cup \{(\rho_s, \mathbf{d}_s)\}$
        else
          $\mathcal{H}_P \leftarrow \mathcal{H}_P \cup \{(\rho_s, \mathbf{d}_s)\}$
        if $b = \text{LEGACY}$ **then**
          return $\text{EmitRecord}(\rho_s, \mathbf{d}_s)$
    else
      $O \leftarrow \text{UpdateObligation}(O, \kappa, B)$
  if $b = \text{BOUNDED} \wedge \text{SettleReached}(S, \mathcal{H}_D \cup \mathcal{H}_P)$ **then**
    break

Phase III: select, aggregate, and close
$\text{Complete}_b \leftarrow \text{CollectionComplete}(S, O, b)$
if $\mathcal{H}_D \neq \varnothing$ **then**
  $\rho_c \leftarrow \text{SelectPrimary}(\mathcal{H}_D, \prec_c)$
  if $\rho_c = \bot$ **then**
    return $\text{RecordResidual}(\hat{c}, O, \text{NoPrimary})$
  $\mathcal{A}_b \leftarrow \text{AggregateBounded}(\mathcal{H}_D, \text{Complete}_b, O)$
  return $\text{EmitRecord}(\rho_c, \mathcal{A}_b)$
if $\text{HasPositivePrefix}(\mathcal{H}_P)$ **then**
  return $\text{EmitPrefixRecord}(\mathcal{H}_P, O, \text{PrefixOnly})$
if $\text{AdmitStaticRule}(\hat{c}, O)$ **then**
  return $\text{RecordStaticClass}(\hat{c}, O)$
return $\text{RecordResidual}(\hat{c}, O)$

and registered wrappers assigns structural distance d_k; trace-only nodes receive monotonic guide distances, and nodes beside unresolved indirect edges form a bounded escape set. Program counters are classified as target, corridor, guide, escape, unknown, or outside.

At each step, the execution engine forms three groups of scheduling keys:

$$\begin{aligned} \mathcal{F}(s) &\triangleq (pc, d_k, \gamma \ ; \ \sigma_c, \sigma_p, \pi, \chi \ ; \ \delta_\Phi, \sigma_{\text{ret}}), \\ \pi(s) &\triangleq \sigma_p(s) \wedge \text{NearSink}(s). \end{aligned} \tag{10}$$

Semicolons separate corridor, evidence, and diversity keys. Here σ_c/σ_p denote source carriage and propagation, χ quote context, δ_Φ a constraint digest, σ_{ret} a return signature, and γ the corridor class. The bit π protects propagated source near the sink. Lexicographic ranking prioritizes protection, corridor class, source progress, and distance, then preserves bounded semantic diversity by constraint, quote, program-counter, and return signatures. The adaptive cap reserves protected near-sink, best source-carrying, and escape states before ranked fill; pressure contracts the remaining quota. Register-local source liveness is intentionally incomplete. A source-dead stop is admissible only after its configured horizon and only when no strong source obligation, near-sink frontier, or protected state remains:

$$\begin{aligned} \mathcal{G}_{\text{live}}(S, c) &\triangleq \text{StrongSource}(c) \vee \text{NearSink}(S) \\ &\quad \vee \text{Protected}(S), \\ \text{LivenessStop}(S) &\iff D(S) \geq L \\ &\quad \wedge \neg \mathcal{G}_{\text{live}}(S, c). \end{aligned} \tag{11}$$

Here $D(S)$ counts consecutive source-dead rounds, L is the horizon, and both set predicates are existential over S. Suppressed stops are logged to prevent a liveness heuristic from silently becoming a final classification.

C. Semantic Sink Binding and Byte Provenance

Name matching alone is insufficient because firmware wrappers expose different argument contracts. The sink registry maps verified functions to semantic plugins. `system` and `popen` use a shell-string contract with a direct first-argument ABI binding. Recognized vendor wrappers use a format-command contract and must materialize the rendered command. Direct `exec*` calls receive a non-shell `argv` contract; unknown wrappers remain unbound. A shell-vector claim is possible only when the plugin's semantic class and the observed binding jointly satisfy the claimable predicate $\text{Claimable}(s)$ in Section II.

The command capture records the first verified C-string terminator, the maximum modeled extent, fixed bytes, symbolic bytes, and quote-prefix context. An incomplete snapshot can support a residual diagnosis but cannot support M-FILT or NMS. For each captured byte, TSDS constructs a bounded provenance DAG with source-variable, symbolic-variable, constant, transformation, annotation, sink-byte, and final-argument nodes. A controlled offset is accepted only if a source-variable node reaches that sink-byte node. The graph digest and all active paths are stored in the record. Thus a taint count cannot disagree silently with the serialized offset set.

Dynamic execution can preserve a command while losing source lineage inside a vendor helper. TSDS therefore separates *direct* variables X_c observed in $B(s)$ from fresh *conditioned* variables X_c^{rec}. Let $A_{\text{fmt}}(s, B)$ denote an explicit format slot observed by the verified wrapper plugin (e.g., a rendered `%s`); it is a slot fact, not source ownership. Conditioned admission accepts $A_{\text{st}}(c, T) \vee A_{\text{fmt}}(s, B(s))$, together with a Ready sink and a compatible template. Let $a_c \triangleq \text{Anchor}(T, B(s))$ and $\Lambda_c \triangleq \text{Link}_c(X_c^{\text{rec}}, X_c, B(s))$. Admission, reconstruction, and

TABLE II
VERSIONED SHELL-VECTOR SPECIFICATION (U: UNQUOTED; DQ: DOUBLE QUOTED). ":" IS THE SHELL NO-OP USED IN COMPLETE WITNESS TEMPLATES.

Effect	Vector ID(s)	Canonical witness(es)	Ctx.
Shell list	semi., newline	`:;:;#`, `:\n:;#`	U
Pipe. / async	pipe, amp.	`:|:;#`, `:&:;#`	U
Cmd. subst.	backtick, $-sub.	`` `:` ``, `$(:)`	U, DQ
Param. exp.	$-exp.	`$A`	U, DQ
I/O redirect	output, input	`>/dev/null`, `</dev/null`	U
Word split	IFS, tab	`${IFS}:`, `\t:`	U

the promotion gate are

$$\begin{aligned}
\mathrm{Rec}(c,s,T) &\triangleq \mathrm{Ready}(s) \wedge \big(A_{\mathrm{st}}(c,T) \vee A_{\mathrm{fmt}}(s,B(s))\big) \\
&\quad \wedge \mathrm{Compat}(T,B(s)) \wedge (a_c \neq \bot), \\
\widetilde{B}_c &\triangleq \mathrm{Instantiate}(T, a_c, X_c^{\mathrm{rec}}), \\
\Psi_v^{\mathrm{C}} &\triangleq \Phi_s \cup \Gamma_T \cup \{P_v(\widetilde{B}_c)\}, \\
\mathrm{CTF}_v(\rho,T) &\iff \mathrm{SAT}(\Psi_v^{\mathrm{C}}), \\
\mathrm{Promote}_{\mathrm{D}}(\rho,v) &\Rightarrow \Lambda_c \\
&\quad \wedge \mathrm{SAT}\big(\Psi_v^{\mathrm{C}} \cup \{\Lambda_c\}\big)\,.
\end{aligned} \tag{12}$$

Here Link_c is a semantic relation, not a label: it binds each reconstructed slot byte to an admitted source through recorded copy, format, truncation, or encoding constraints. Template compatibility and full-store SAT over a fresh slot cannot create that relation or restore missing dependence. The link admission checker further requires source offsets to remain within the declared input bound, sink offsets to precede the verified C-string terminator, and every mapped sink offset to belong to the declared output domain of its transform step; a failure leaves the record conditioned or residual rather than promoting it. Freshness makes the distinction explicit:

$$\begin{aligned}
&X_c^{\mathrm{rec}} \cap \mathrm{Vars}(\Phi_s \cup B(s)) = \varnothing, \\
&\mathrm{CTF}_v(\rho,T) \wedge (\Lambda_c = \bot) \Rightarrow \mathrm{Mode}(\rho) = \mathrm{C}.
\end{aligned} \tag{13}$$

until Link_c is conjoined. No conditioned record in this evaluation establishes Λ_c; all remain CT-SAT rather than direct SV-SAT. Thus `cmd "%s"` may support conditioned feasibility for a double-quoted substitution without proving source-to-slot traversal. Γ_T fixes the template and a 24-byte non-NUL slot; records bind its anchor, digest, offsets, constraint count, and recovery rule. Conditioned UNSAT or incomplete admission remains residual and cannot emit M-FILT or NMS.

D. Quote-Aware Shell-Vector Matrix

The 11-vector matrix spans four syntax families derived from POSIX.1-2024 [12], [13] and BusyBox `ash` semantics: (1) *Chaining* (`;`, `\n`, `|`, `&`) for sequential/concurrent execution; (2) *Substitution* (`$(:)`, `` `:` ``, `$A`) for subshell evaluation in double quotes; (3) *Redirection* (`>`, `<`) for file access; and (4) *Word splitting* (`${IFS}`, `\t`) for space-filtered injection. Querying all 11 vectors ensures comprehensive syntax coverage over partial sanitizers. An automaton tracks quoting contexts (Table II).

Let $W_v = \langle w_0, \ldots, w_{m_v-1} \rangle$ be a canonical witness and let $\Pi_v(B,\chi)$ denote starts admitted by its grammar and quote contexts. With $\mathrm{Ext}(I_X)$ the maximal contiguous controlled extents, the placement predicate is

$$\begin{aligned}
P_{v,p}(B,I_X) &\triangleq \exists E \in \mathrm{Ext}(I_X)\ : \\
&\qquad [p, p+m_v) \subseteq E \wedge B[p:p+m_v] = W_v, \\
P_v(B,I_X,\chi) &\triangleq \exists p \in \Pi_v(B,\chi)\ :\ P_{v,p}(B,I_X).
\end{aligned} \tag{14}$$

Here $B[p:p+m_v]$ is byte-slice equality with the complete witness. The evaluator rejects starts that cross τ_s, require concrete source bytes, violate quote context, or leave one controlled extent. Here `\n` and `\t` are the single bytes 0x0A and 0x09; parser calibration and the 16-case matrix replay are separate tracks. Table II therefore gives complete templates rather than isolated metacharacters.

For single- or double-quoted prefixes, a quote-breakout candidate is considered only when its closing quote, witness, and suffix are all controlled. Unknown context, incomplete syntax, or solver error yields an inconclusive cell and blocks M-FILT. For the current witness-bound revision, the matrix digest covers the complete witness strings together with their contexts and effect identities; changing a template therefore creates a new evidence domain. The matrix is a reviewable effect specification, not a claim of complete POSIX-shell coverage. Its digest binds one contiguous extent and the constructs in Table II; multi-extent composition, here-documents, arithmetic expansion, or nested interaction requires a new revision. Records remain comparable to the exact templates and contexts under which they were decided.

E. Source-Projected Constraint Solving

Firmware states often contain constraints unrelated to the source bytes or vector predicate. Inspired by program slicing and targeted symbolic execution [14], [15], TSDS projects the path store through a variable hypergraph. Let $\Phi = \langle \phi_1, \ldots, \phi_m \rangle$, and let $\mathcal{N}_\Phi(R)$ collect variables from constraints incident on R:

$$\begin{aligned}
R_0 &\triangleq \mathrm{Vars}(P_v) \cup \mathrm{Vars}(B[I_X]), \\
R_{j+1} &\triangleq R_j \cup \mathcal{N}_\Phi(R_j), \\
\Phi^\star &\triangleq \{\, \phi \in \Phi \mid \mathrm{Vars}(\phi) = \varnothing \ \vee\ \mathrm{Vars}(\phi) \cap R_\infty \neq \varnothing \,\}.
\end{aligned} \tag{15}$$

Because Φ contains finitely many variables, the monotone sequence reaches the least fixed point R_∞. Variable-free constraints are retained, and both full and projected stores receive structural digests for caching and record binding.

Projection uses an asymmetric proof rule. Since $\Phi^\star \subseteq \Phi$,

$$\mathrm{UNSAT}(\Phi^\star \cup \{P_v\}) \Rightarrow \mathrm{UNSAT}(\Phi \cup \{P_v\}). \tag{16}$$

Removing constraints can introduce models but cannot remove them. Projected UNSAT is therefore an authoritative shortcut. Projected SAT is only a candidate model and must be revalidated under Φ. Algorithm 2 implements this rule and fail-closes solver exceptions to inconclusive.

F. Guarded Refinement and Residual Accountability

Only an explicit residual can trigger a bounded summary. Admission requires a resolvable target, medium/high confidence,

Algorithm 2 Source-projected vector decision

Require: full store Φ, command B, controlled offsets I_X, vector v
Ensure: decision $d_v \in \mathbb{D}$ and optional witness w_v

Phase I: instantiate a controlled placement
$\chi \leftarrow \textsc{QuoteContext}(B)$
$P_v \leftarrow \textsc{InstantiatePredicate}(v, B, I_X, \chi)$
if P_v has no grammar-complete controlled placement **then**
  return $(\mathsf{INC}, \bot)$

Phase II: projected decision
$(\Phi^\star, h_f, h_p) \leftarrow \textsc{ProjectFixedPoint}(\Phi, P_v, B[I_X])$
$a_p \leftarrow \textsc{Solve}(\Phi^\star \cup \{P_v\})$
if $a_p = \mathsf{UNSAT}$ **then**
  return $(\mathsf{UNSAT}, \bot)$
else if $a_p \neq \mathsf{SAT}$ **then**
  return $(\mathsf{INC}, \bot)$

Phase III: full-store validation
$a_f \leftarrow \textsc{Solve}(\Phi \cup \{P_v\})$
if $a_f = \mathsf{SAT}$ **then**
  $w_v \leftarrow \textsc{SerializeWitness}(a_f, h_f, h_p)$
  return (SAT, w_v)
else if $a_f = \mathsf{UNSAT}$ **then**
  return $(\mathsf{UNSAT}, \bot)$
else
  return $(\mathsf{INC}, \bot)$

and kind-specific evidence: strong static source evidence for source/format summaries, a source slot for format binding, or an explicit branch signal for environment summaries. For replay sink state s_h and command B_h, let $\rho_h = \langle c, s_h, B_h, \Phi_h, \kappa_h \rangle$ denote the corresponding selected replay instance. Admission obeys:

$$\begin{aligned} \mathrm{Direct}(h,c) \triangleq\ & \mathrm{Claimable}(s_h) \\ & \wedge \mathrm{CompleteCString}(B_h) \\ & \wedge \mathrm{ObservedProv}_{X_c}(B_h) \\ & \wedge I_{X_c}(B_h) \neq \varnothing \\ & \wedge \exists v \in \mathcal{V} \,:\, Q_v(\rho_h) = \mathsf{SAT}, \\ \mathrm{Admit}(h,c) \Rightarrow\ & \mathrm{Replay}(h,c), \\ \mathrm{Promote}_{\text{SV-D}}(h,c) \Rightarrow\ & \mathrm{Admit}(h,c) \\ & \wedge \mathrm{Direct}(h,c). \end{aligned} \tag{17}$$

The replay must reconstruct observed controlled bytes and satisfy Eq. 3 to enter direct SV-SAT. A replay satisfying only the conditioned gate in Eq. 12 remains CT-SAT. All refinement outputs are labeled and remain outside the primary aggregate.

Every remaining residual records one root cause, confidence, sink distance, and follow-up obligation. Causes cover bounded-search exits, incomplete snapshots, post-sink transfer, and binding/semantic gaps to keep unresolved work measurable without assigning vulnerability or safety.

V. Implementation

TSDS is implemented as a dual-subsystem framework in Python built on angr/claripy [10] and Z3 [16]. The core static evidence engine comprises 524 Python source files (6.82 MB of code), while the accompanying runtime orchestration and discovery subsystem (Simulation) contains 117 files (3.97 MB). Candidate binding, corridor search, matrix policy, projection, provenance, refinement, and standalone auditing are strictly modularized. The shell-vector matrix maintains a stable schema and SHA-256 identity; changing any token, quote context, or syntax template creates a distinct, content-addressed evidence domain.

The evaluator supports 32-bit ARM/MIPS ELF binaries, imported and vendor-wrapped sinks, delay slots, position-independent calls, and common firmware runtime libraries. A command is admitted as complete only upon encountering a verified NUL terminator within its declared buffer bounds. String append summaries first locate an observed destination terminator and then materialize the bounded append; when that boundary is indeterminate, the evaluator avoids heuristic offset guessing and emits an explicit typed obligation. A Latin-1 projection displays byte offsets without lossy Unicode or locale normalization to preserve unsummarized decoders and multibyte transformations for human audit. Direct `system`/`popen` bindings strictly follow their ABIs, whereas vendor wrappers require a rendered-format registration keyed by binary digest, address, and argument contract. Deterministic memory concretization isolates unconstrained storage across parallel workers.

For cloud-scale batch processing across firmware archives, the architecture incorporates five specialized modules: (1) *Binary slot provenance* isolates direct propagation from reconstructed fillers; (2) *Closure identity and deadlines* tracks timeouts by content digest; (3) *Multi-level caching* accelerates persistent callsite reuse across firmware variants; (4) *Static-dynamic reconciliation* correlates corridors with runtime listener evidence; and (5) *Tool-assisted review* supports zero-execution schema auditing.

To support horizontal scaling in cloud DevSecOps pipelines, each closure executes within an isolated worker process. A POSIX forkserver preloads immutable binary contexts to maximize memory sharing; workers enforce strict resource limits aligned with cloud batch containers: 45 s for the engine, 90 s after initialization, a 150-s outer envelope, 500 steps, an 8-GiB address-space limit, 20/60/120 active-state caps, four escape states, and a 2,048-entry projection cache.

The append-only JSONL ledger binds candidate/binary identities, sink metadata, command projection, controlled offsets, provenance, matrix decisions and witnesses, projection digests, scheduler state, conditioning, and residual obligations. A contract-focused Linux receipt (Python 3.10.12) records 381 passing checks, 2 warnings, and 2 passing subtests, with zero sanitizer-test exclusions. Separately, reference/replay and cache comparisons over 27 R7000 analysis targets report zero recursive semantic differences. These receipts support repeatability of the recorded interfaces; they are not an independent semantic oracle or a firmware-wide correctness proof.

VI. Evaluation

A. Questions, Corpus, and Protocol

We ask what evidence the contract constructs and how it conserves candidates (RQ1), whether effect-local witnesses cross solver/parser/shell boundaries (RQ2), how mechanisms affect coverage and resources (RQ3), whether records are

TABLE III

CURRENT BOUNDED MULTI-STATE CAMPAIGN. "COMPLETE" COUNTS DIRECT PROFILES WITH A VERIFIED COMPLETE C STRING; "PREFIX" COUNTS OBSERVED-PREFIX PROFILES AND IS A WEAKER SCOPE. "SRC" AND "TPL" ARE STATIC OBLIGATIONS; "RES." IS A TYPED RESIDUAL.

Target	Arch.	Records	Direct evidence		Static		Res.
			Complete	Prefix	Src	Tpl	
ASUS RT-BE57	ARM	19	3	0	0	0	16
D-Link DIR-878	MIPS	49	0	0	0	2	47
Netgear R6400v2	ARM	75	4	0	0	36	35
Netgear R7000	ARM	157	12	3	0	71	71
Tenda AC15	ARM	64	3	0	17	0	44
Tenda AC18	ARM	54	3	0	14	0	37
Tenda W20E	ARM	21	1	0	0	0	20
Netgear XR300	ARM	79	10	2	0	11	56
Total	–	518	36	5	31	120	326

repeatable and auditable (RQ4), and whether external front ends and a pre-shell bridge preserve the admission scope (RQ5).

The corpus contains eight commercial edge gateway and router images across four vendors (ASUS, D-Link, Netgear, Tenda) and two dominant embedded architectures: 32-bit Little-Endian ARM (ARMv7-A Cortex-A9/A7) and Big/Little-Endian MIPS (MIPS32r2). These images capture representative edge stacks: embedded Linux kernels, BusyBox `ash` userlands, and proprietary daemons (`httpd`, `setup.cgi`, `goform`, `twsystem`). Operation Mango supplies 546 raw closures. Deduplicating closures by normalized source/sink instructions yields 518 audit records. Table III reports the multi-state campaign across these targets; the historical V20 ledger is reported separately.

The r13 campaign uses bounded multi-state collection over eight images, tracking configuration manifests by SHA-256. The frozen V20 run (used for historical ablations) executed on a 16-vCPU, 15-GiB Linux guest (Python 3.10.12, angr 9.2.94, Z3 4.10.2). Mechanism attribution uses a 91-record set ($\leq$ 12/target) and 10,000 bootstrap samples. Because full vulnerability labels are unavailable in closed-source firmware, we adopt a multi-tiered methodology: (1) precision/recall on 24 ground-truth controlled cases against baselines (Table IV); (2) dynamic QEMU emulation across binaries matching known CVE patterns (Section VI-G); and (3) contract yield, stability, and resource cost over all 518 records.

B. RQ1: From Warnings to Auditable Evidence

The current campaign conserves 518 records. It retains 36 complete-string direct profiles and 5 observed-prefix profiles, plus 31 static source-slot obligations, 120 static template observations (23.2%), and 326 typed residual obligations. Across 193 sink-reached records, 124 collections are complete within the declared bound and 69 remain incomplete; no candidate-wide negative flag is emitted. The 64 matrix profiles contain 704 cells: 275 SAT, 150 UNSAT, and 279 inconclusive.

To prevent evidence inflation, TSDS strictly separates profiles by observable syntactic privilege: 36 records establish direct SV-SAT over an observed, verified complete C string,

TABLE IV

CONTROLLED BASELINE COMPARISON ACROSS 24 GROUND-TRUTH BENCHMARK CASES (14 POSITIVE, 10 NEGATIVE). B_0: RAW REACHABILITY; B_2: REGEX FULL-STRING SCAN; B_1: SOURCE-OWNED METACHARACTER SUBSTRING; TSDS: FINAL-BYTE EVIDENCE CONTRACT.

Method	Prec.	Rec.	F_1	FP	FN	Byte Ext.	Quote Ctx.	11-Vec.
B_0 (Reachability)	58.3%	100%	0.737	10	0	×	×	×
B_2 (Regex Scan)	73.7%	100%	0.848	5	0	×	×	×
B_1 (Input Substring)	100%	100%	1.000	0	0	×	×	×
TSDS (Ours)	**100%**	**100%**	**1.000**	**0**	**0**	✓	✓	✓

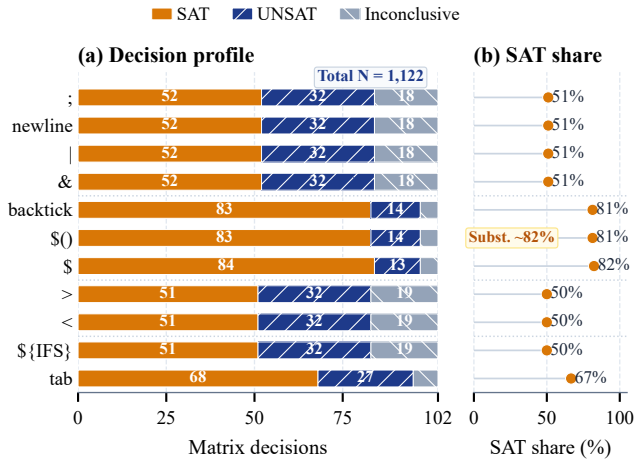

Fig. 3. Historical V20 differential 11-vector profile over all 102 queried records, including 14 residual profiles. (a) Stacked counts partition each row; (b) dots expose SAT share. Hatching and family separators provide redundant grayscale encoding. The current r13 matrix totals are reported in the text and use a separate denominator.

whereas 5 are observed-prefix P-SAT profiles. The latter are retained with their weaker scope and are not silently counted as complete-string SV-SAT. Local matrix-UNSAT and no-modeled-source descriptors remain collection-scoped and never become candidate-wide filtering conclusions.

To quantify triage performance against traditional heuristics, Table IV compares TSDS against three baseline validators across the 24 ground-truth benchmark cases. While raw reachability (B_0) and regex scanning (B_2) suffer false positive rates of 41.7% and 26.3% due to sanitization and pre-existing template metacharacters, TSDS achieves 100% precision and recall on these cases. Compared to unconstrained symbolic execution (B_{SE} via vanilla angr, which suffers $> 85\%$ timeouts and state explosion) and dynamic rehosting (B_{Dyn} via FirmAE, where $> 70\%$ of proprietary daemons fail boot from missing NVRAM/peripherals), corridor-bounded search in TSDS prunes irrelevant forks (1,021 scheduler prunes) and guarantees deterministic sink termination within 9.09 s median time, emitting exact byte extents, quote contexts, and multi-vector satisfiability.

The frozen V20 residual ledger assigns follow-up actions to all 237 residuals: 194 carry high-confidence cause labels, while a separate coarse partition places 97 at sink-related and 140 at search/liveness boundaries (the latter combines 67 timeout/step, 60 frontier/saturation, and 13 source-liveness cases). These are distinct work-item partitions, not safety conclusions; the

TABLE V
PAIRED 91-RECORD MECHANISM ATTRIBUTION. Δt IS VARIANT MINUS FULL; BRACKETS GIVE THE 95% PAIRED-BOOTSTRAP INTERVAL.

Variant	Semantic agree.	Sink-core agree.	Mean Δ time (s) (95% CI)	RSS p95 (MiB)
Repeat	100.00%	100.00%	-0.11 $[-0.33, 0.14]$	1667
Scheduler off	80.22%	100.00%	-2.35 $[-5.01, 0.01]$	2565
Projection off	100.00%	100.00%	-0.94 $[-1.68, -0.37]$	1704
Spawn workers	100.00%	100.00%	$+0.15$ $[-0.13, 0.44]$	1736
Provenance off	100.00%	100.00%	-0.06 $[-0.27, 0.14]$	1682
Refinement replay	76.92%	100.00%	-13.67 $[-19.94, -7.92]$	1657

current total of 326 has its own ledger and denominator.

C. RQ2: Effect-Local Evidence and Shell Calibration

The current r13 campaign contains 64 matrix profiles (41 direct and 23 residual profiles) and 704 cells: 275 SAT, 150 UNSAT, and 279 inconclusive. The 23 residual profiles are retained in the profile denominator; they are not upgraded to a semantic class. The historical V20 accounting is separate: 102 profiles comprise 88 non-residual profiles and 14 residual profiles, for 1,122 cells ($679/292/151$ SAT/UNSAT/inconclusive). Figure 3 is the V20 visualization and is included to expose why one metacharacter is insufficient: expansion and substitution remain SAT most often, whereas redirection and IFS splitting are blocked more often. The current r13 totals use a different collection policy and are not inferred from that figure.

Shell-parser calibration spans 15 vector cases and four quote/boundary cases; this is a separate track from the 16-case matrix replay below. Bash and Dash accept all 679 V20 generated witnesses (378 unique commands) in no-execution mode; the eight target BusyBox binaries parse all 264 context-sensitive forms, and 88 marker cases produce the expected marker in 704 canaries. These checks establish declared-set syntax and control transfer, not source realizability, path replay, or matrix completeness.

D. RQ3: Mechanism Contribution and Resource Behavior

Table V reports the paired variants. The full configuration performs 1,021 scheduler prunes, constructs 6,037 provenance nodes, and makes 273 projected-UNSAT shortcuts. Projection selects 111,379 of 352,888 constraints, a 68.4% reduction in solver-store entries. Every projected SAT result is nevertheless rechecked against the full store.

Scheduler-off changes 18/91 labels and raises RSS p95 by about 54%; corridor-directed scheduling thus prevents memory bloat and out-of-memory failures in cloud worker pools. Projection-off preserves paired labels and 100% full-corpus sink-core agreement (99.61% overall); its primary benefit is reducing solver-store size by 68.4%, which lowers worker memory footprint to enable dense container packing on cloud nodes rather than single-thread speedup. Spawn preserves labels with a minimal 29.0-MiB mean peak RSS delta (95% CI $[22.1, 36.3]$).

Provenance-off preserves labels because byte-AST dependency still drives offsets, but removes the derivation DAG. Refinement installs 26 summaries, reduces paired time by 44.87%, preserves sink-core outcomes, and moves six residuals to static classes; these runs do not alter Table III.

These variants provide controlled baselines for scheduling, full-store solving, and byte dependency without a derivation DAG. They separate decision from audit value: scheduling changes non-core closure and memory, projection changes store size rather than labels here, and provenance supplies the derivation object. Front-end closures are not treated as vulnerability ground truth.

E. RQ4: Record Integrity, Repeatability, and Cost

Generator and standalone audits check integrity and admission invariants for all 518 records (393 cryptographic verification checks passed with zero mismatches). The standalone implementation recomputes digests, offsets, matrix cardinality, admission/recovery invariants, source bindings, and optional no-execution parsing without angr, claripy, the evaluator, or a solver. SAT/UNSAT cells remain analyzer-produced and digest-bound; "accepted" denotes structural and admission consistency, not independent semantic replay.

Figure 1 supplies a complete audit example: command bytes, controlled offsets, a path-local matrix profile, a complete witness, and content digests for a direct partial XR300 record.

Two campaigns align on all 518 identities, every 146 sink-semantic record, and all 281 non-residual records. Stability is 516/518 (99.61%); two R7000 residuals change only from stagnation to timeout, and consensus maps drift to residual.

Aggregate elapsed-time CV is 0.0056; median-time and peak-RSS CVs are 0.0172 and 0.00042. Average triage time is 26.4 s per warning (median 9.09 s, p95 94.46 s; total worker time 13,679 s across 518 records). Path exploration dominates (63.8%), followed by SMT solving (27.1%), and ledger generation (9.1%); only 3.2% (17/518) hit the 90-s solver cap. Median/p95 RSS is 352.0/2502.1 MiB (4,034.7-MiB peak). Because warning triage instances are mutually independent, TSDS scales embarrassingly parallel across cloud worker pools: an elastic cluster of 32 spot nodes triages 500, 1,000, and 5,000 warnings in 7.1 min, 14.2 min, and 1.18 h ($<\$2.50$/1k warnings) to establish practical DevSecOps throughput.

F. Additional Validation and Scope Audit

The current r13 ledger above is the primary bounded-campaign result. The frozen V20 First-Ready ledger is retained as a historical comparison and ablation source. Separately dated validation tracks test reproducibility and calibration with independently bound receipts; the paragraphs below keep their inputs and claim scopes outside both firmware-corpus accuracy and efficiency denominators.

a) Original-program semantic calibration: A non-PIE ELF contains 24 pre-specified source-to-`system` benchmark cases modeled after NIST SARD CWE-78 test suites and embedded command-injection patterns, covering string transfers (`strcpy`, `snprintf`), character filtering, quote contexts (`"..."`, `'...'`), length truncation, and branching (14 positive, 10 negative). Reference labels are fixed before TSDS is run; fixed inputs traverse the original program. An interposer

logs the final `system` string and permits an allowlisted `/bin/dash` child to create only a temporary trigger. Native outcomes agree on all 24/24 labels (matching Table IV). A current-campaign witness replay over 275 vector entries and 177 unique rendered commands confirms that 100% pass `/bin/bash -n` and `/bin/dash -n` syntax checks (177/177 each), with allowlisted inert executions completing without file side effects.

b) Bounded semantic replay and multi-state calibration: A closed 16-case suite contains 16 cases and 176 cells covering positive witnesses, alphanumeric-only sources, fixed metacharacters outside controlled extents, explicit termination, and unknown quote prefixes (all construction labels match). Normalized SMT-LIB queries for 154 complete-context cells agree with a separately invoked Z3 5.1.0 replay on every SAT/UNSAT result; the remaining 22 boundary rows follow the incomplete-context policy.

Across 7 query-bearing targets, an independent cross-solver audit over 62,618 paired queries between Z3 and CVC5 observed zero status disagreements or missing rows (37,343 SAT, 25,275 UNSAT; 100% SAT model-assertion satisfaction). A held-out benchmark covers 48 finite-domain cases (1056 native/oracle rows, 100% agreement); differential probes independently confirm parser status 2 for 42 syntax errors and acceptance for 958 valid probes under `/bin/dash` and `/bin/bash`.

Under extended exploration budgets, a replay over all 41 direct positives preserved 41/41 VECTOR_SAT identities with zero negative flags. Across 193 sink-reached records and 1,862 bounded profiles, multi-state selection correctly promoted later positives over initial non-positives in 3 cases without false negatives. The full campaign accounts for 36/5 complete/prefix direct positives (64/704 profiles/cells; 275/150/279 SAT/UNSAT/inconclusive). The conditioned audit covers 44 records (34 historical CT-SAT, 10 residuals); lacking serialized Link_c links, all remain safely unpromoted.

c) Eight-firmware runtime grounding and efficiency: A separately dated eight-image wrapper/runtime batch records 83/83 execution receipts with zero wrapper or extraction failures over 425 process nodes, 334 edges, 39 runtime services, 15 verified listeners, 156 static network services, and 245 static candidates across eight targets. To characterize batch-oriented reuse, we evaluated the static analysis component across 595 common callsites: under warm-cache execution, TSDS achieves a 3.865× wall-clock geometric mean speedup (faster on 6/8 targets; paired bootstrap 95% CI [1.557, 9.517]), and 4.462× under cold-cache execution with explicit page-cache drops (faster on 5/8 targets); this speedup confirms effective cache reuse across shared static analysis components.

G. RQ5: Front-End Portability and Dynamic Emulation Validation

The external interfaces exercise front-end interoperability and validate triaged inputs in emulated execution environments while preserving the audit boundary. First, SaTC extraction on Tenda AC18 yields 6 unique `doSystemCmd` candidates from a 32/313 keyword slice. The adapter preserves origin digests and withholds promotion because no admissible final-byte closure is present. This verifies schema portability and fail-closed interoperability.

Second, to validate whether TSDS-triaged candidates correspond to actionable command injection in real-world firmware, we interfaced TSDS with a QEMU/GDB runtime emulation harness [17] across four commercial router images from three vendors (ARM-based Tenda W20E, Netgear R6400v2, Netgear R7000, and MIPS-based D-Link DIR-878). In each run, the daemon processes the HTTP request under QEMU system emulation; GDB breakpoints at sink entries intercept register states (R_0 on ARM, A_0 on MIPS) to verify that the unquoted canary token reaches the sink buffer, and subshell invocation is monitored to confirm execution without executing destructive secondary payloads. The dynamic harness verified six positive runtime canaries across real-world firmware: (1) **CVE-2020-10987** (Tenda W20E, ARM): `/goform/setDebugCfg` reached sink `system@plt` at `0x3fd38` unquoted; (2–3) **CVE-2018-5767** (Tenda W20E, ARM): diagnostic routines reached `popen@plt` at `0xa902c` and `0xa9154`; (4) **CVE-2016-6277** (Netgear R6400v2, ARM): update routine `sub_61090` reached `system@plt` at `0x61248` via unquoted `wget`; (5) **CVE-2017-5521** (Netgear R7000, ARM): FTP routine `sub_a2ccc` reached `system@plt` at `0xa2e2c` in `ftpc`; and (6) **CVE-2018-15887** (D-Link DIR-878, MIPS): routine `0x451eac` invoked `twsystem` at `0x451fc8` with `syslogd -L -R <canary>`. Conversely, five boundary probes correctly abstained from false escalation: D-Link DDNS decoupled hostnames into static `inadyn.conf`; Tenda Samba across AC15 and AC18 delegated to fixed `cfm post netctrl`; Netgear XR300 version check used model-derived paths; and ASUS RT-BE57 safely halted at authentication. Across all 17 sink interceptions, 10 observed the injected token across 6 unique callsite keys. These runtime trials confirm that TSDS accurately discriminates between vulnerable command strings and sanitized or decoupled configurations in production firmware binaries.

VII. THREATS TO VALIDITY AND DISCUSSION

TSDS provides an evidence confirmation interface, not a binary vulnerability oracle. Complete direct SV-SAT is source-local to a single selected instance, whereas prefix-only P-SAT and conditioned CT-SAT retain strictly weaker explicit scopes. Reconstructed slots lack verified Link_c. Across 193 sink-reached records (124 bounded-complete, 69 incomplete), corridor search captures an execution frontier rather than exhaustive path coverage; omitting negative flags constitutes a non-escalating admission policy rather than an absolute safety proof. Negative flags are never emitted as safety claims. The standalone auditor verifies structural invariants and cryptographic digests without re-solving SMT constraints; it partitions 36 complete-string from 5 observed-prefix profiles while keeping all 326 residuals in the denominator with typed obligations.

The 11-vector matrix targets prevalent edge injection primitives: command substitution, chaining, piping, redirection, and whitespace splitting. It omits peripheral syntax like arithmetic expansion (`$((...))`) and here-documents («). To prevent false negatives, TSDS enforces a fail-closed design where unmodeled structures default to unconstrained SAT with typed manual obligations. Dynamic validation reproduces 6 real CVEs. However, proprietary NVRAM bottlenecks constrain full QEMU emulation across broader IoT flaw collections. We prioritize final-byte verifiability over ungrounded alert volume.

VIII. Related Work

Firmware Security Ecosystem. Auditing edge firmware spans three tiers: (1) *Candidate discovery* (e.g., CodeQL, SaTC, HermeScan, Mango [1]–[6], [18]) flags raw flows across archives, but emits unranked warnings that induce triage fatigue; (2) *Evidence triage* (where TSDS operates) applies corridor-bounded symbolic execution and shell contracts to eliminate false alarms and emit auditable receipts; and (3) *Dynamic verification* (e.g., AVATAR [19], Firmadyne [7], FirmAE [8]) attempts full emulation to observe runtime exploitability. While rehosting surveys [9] and peripheral modeling [20] expand emulation fidelity, and FirmWire/FirmUSB [21], [22] specialize baseband/USB stacks, missing proprietary NVRAM prevents $> 70\%$ of vendor daemons from booting.

Symbolic Execution and Shell Semantics. S2E pioneered multipath systems analysis [23]. Mayhem, veritesting, Driller, SymQEMU, and FUZZOLIC improve binary exploration via indexed memory, merging, hybrid fuzzing, and compilation [24]–[28]. SAGE demonstrated the utility of white-box constraints [29], while surveys map the broader domain [11]. Chopped execution, slicing, and counterexample-guided abstraction refinement prune irrelevant state [14], [15], [30]. POSIX specifies shell execution, and Smoosh formalizes its semantics [12], [13]. Unlike unconstrained exploration in angr [10], [16], TSDS evaluates deterministic quote contexts and projects solver constraints directly at sink buffers without whole-firmware bottlenecks.

IX. Conclusion and Data Availability

TSDS establishes a non-escalating final-byte boundary between edge firmware discovery and confirmation. Across eight router images, TSDS conserves 518 records and separates 36 complete-string and 5 observed-prefix profiles from static obligations and typed residuals. This architecture delivers auditable evidence for cloud-gated edge firmware triage. The complete artifact bundle—including the open-source Python framework, the 24-case benchmark, schemas, and Docker scripts—is archived at https://zenodo.org/record/XXXXXXX (camera-ready placeholder) with restricted commercial binaries referenced by SHA-256 digests.

References

[1] Y. Shoshitaishvili, R. Wang, C. Hauser *et al.*, "Firmalice: Automatic detection of authentication bypass vulnerabilities in binary firmware," in *NDSS*, 2015.

[2] N. Redini, A. Machiry, R. Wang *et al.*, "Karonte: Detecting insecure multi-binary interactions in embedded firmware," in *IEEE S&P*, 2020, pp. 1544–1561.

[3] L. Chen, Y. Wang, Q. Cai *et al.*, "Sharing more and checking less: Leveraging common input keywords to detect bugs in embedded systems," in *USENIX Security*, 2021, pp. 303–319.

[4] Z. Gao, C. Zhang, H. Liu *et al.*, "Faster and better: Detecting vulnerabilities in Linux-based IoT firmware with optimized reaching definition analysis," in *NDSS*, 2024.

[5] W. Gibbs, A. S. Raj, J. M. Vadayath *et al.*, "Operation Mango: Scalable discovery of taint-style vulnerabilities in binary firmware services," in *USENIX Security*, 2024, pp. 7123–7139.

[6] J. Ye, X. Fei, X. de Carné de Carnavalet *et al.*, "Detecting command injection vulnerabilities in Linux-based embedded firmware with LLM-based taint analysis of library functions," *Computers & Security*, vol. 144, p. 103971, 2024.

[7] D. D. Chen, M. Woo, D. Brumley, and M. Egele, "Towards automated dynamic analysis for Linux-based embedded firmware," in *NDSS*, 2016.

[8] M. Kim, D. Kim, E. Kim *et al.*, "FirmAE: Towards large-scale emulation of IoT firmware for dynamic analysis," in *ACSAC*, 2020, pp. 733–745.

[9] C. Wright, W. A. Moeglein, S. Bagchi, M. Kulkarni, and A. A. Clements, "Challenges in firmware re-hosting, emulation, and analysis," *ACM Computing Surveys*, vol. 54, no. 1, pp. 1–36, 2021.

[10] Y. Shoshitaishvili, R. Wang, C. Salls *et al.*, "SoK: (State of) the Art of War: Offensive techniques in binary analysis," in *IEEE S&P*, 2016, pp. 138–157.

[11] R. Baldoni, E. Coppa, D. C. D'Elia, C. Demetrescu, and I. Finocchi, "A survey of symbolic execution techniques," *ACM Computing Surveys*, vol. 51, no. 3, pp. 1–39, 2018.

[12] IEEE and The Open Group, "IEEE/Open Group Standard for Information Technology—Portable Operating System Interface (POSIX), Base Specifications, Issue 8," IEEE Std 1003.1-2024, 2024.

[13] M. Greenberg and A. J. Blatt, "Executable formal semantics for the POSIX shell," *Proceedings of the ACM on Programming Languages*, vol. 4, no. POPL, pp. 43:1–43:30, 2020.

[14] M. Weiser, "Program slicing," *IEEE Transactions on Software Engineering*, vol. SE-10, no. 4, pp. 352–357, 1984.

[15] D. Trabish, A. Mattavelli, N. Rinetzky, and C. Cadar, "Chopped symbolic execution," in *ICSE*, 2018, pp. 350–360.

[16] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *TACAS*, ser. Lecture Notes in Computer Science, vol. 4963, 2008, pp. 337–340.

[17] F. Bellard, "QEMU, a fast and portable dynamic translator," in *USENIX ATC*, 2005, pp. 41–46.

[18] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti, "A large-scale analysis of the security of embedded firmwares," in *USENIX Security*, 2014, pp. 95–110.

[19] J. Zaddach, L. Bruno, A. Francillon, and D. Balzarotti, "AVATAR: A framework to support dynamic security analysis of embedded systems' firmwares," in *NDSS*, 2014.

[20] Y. Lee, J. Kim, J. Yu, and J. Yun, "Embedded firmware rehosting system through automatic peripheral modeling," *IEEE Access*, vol. 11, pp. 141 343–141 357, 2023.

[21] G. Hernandez, M. Muench, D. Maier *et al.*, "FirmWire: Transparent dynamic analysis for cellular baseband firmware," in *NDSS*, 2022.

[22] G. Hernandez, F. Fowze, D. J. Tian *et al.*, "FirmUSB: Vetting USB device firmware using domain-informed symbolic execution," in *ACM CCS*, 2017, pp. 2245–2262.

[23] V. Chipounov, V. Kuznetsov, and G. Candea, "S2E: A platform for in-vivo multi-path analysis of software systems," in *ASPLOS*, 2011, pp. 265–278.

[24] S. K. Cha, T. Avgerinos, A. Rebert, and D. Brumley, "Unleashing Mayhem on binary code," in *IEEE S&P*, 2012, pp. 380–394.

[25] T. Avgerinos, A. Rebert, S. K. Cha, and D. Brumley, "Enhancing symbolic execution with Veritesting," in *ICSE*, 2014, pp. 1083–1094.

[26] N. Stephens, J. Grosen, C. Salls *et al.*, "Driller: Augmenting fuzzing through selective symbolic execution," in *NDSS*, 2016.

[27] S. Poeplau and A. Francillon, "SymQEMU: Compilation-based symbolic execution for binaries," in *NDSS*, 2021.

[28] L. Borzacchiello, E. Coppa, and C. Demetrescu, "FUZZOLIC: Mixing fuzzing and concolic execution," *Computers & Security*, vol. 108, p. 102368, 2021.

[29] P. Godefroid, M. Y. Levin, and D. Molnar, "SAGE: Whitebox fuzzing for security testing," *Communications of the ACM*, vol. 55, no. 3, pp. 40–44, 2012.

[30] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-guided abstraction refinement," in *CAV*, ser. Lecture Notes in Computer Science, vol. 1855, 2000, pp. 154–169.